Ticketflow — Day 1

Three tables, and the day the database starts saying no

> **Most of today you already know how to do.** Layers, pool, lifespan, `$1`, `dict(row)`, `APIRouter`, query parameters, `HTTPException` — all unchanged, and none of it gets explained again.
>
> Two things are new, and they are the same idea twice: **the database starts enforcing your rules**, and **two writes learn to happen together or not at all**.

What is new today

NEW	WHY IT SHOWS UP NOW
constraints — `REFERENCES`, `CHECK`, composite `UNIQUE`	Three related tables. Python can no longer be the only thing stopping bad data.
`ON DELETE CASCADE`	Delete an event — what happens to its 400 seats?
transactions	Creating an event is two writes. An event with no seats is garbage.
a `JOIN`	"How many seats are left?" lives in a different table from the event.
`COALESCE`	A `PATCH` where the user sends only the field they want changed.
`generate_series`	Make 400 seat rows without a Python loop.

About seven hours. The schema is the slow part and it is worth being slow about — almost every rule you write there is a rule you then never have to write in Python. Everything after it is project 3 with three tables instead of one.

PART 1 · The three tables, and what they refuse

In project 3 you had one table, and every rule lived in your service file: is this a real URL, is this code free. That worked because there was only one way in.

Now there are three tables that point at each other, and the rules are about *relationships*: a seat belongs to an event, a booking belongs to a seat. If those rules live only in Python, then one forgotten check — or one `psql` session where you fix something by hand — and your data is quietly wrong forever.

So this time the rules go in the schema, and Postgres enforces them for every route in and out.

Events

schema.sql

```
DROP TABLE IF EXISTS bookings;
DROP TABLE IF EXISTS seats;
DROP TABLE IF EXISTS events;

CREATE TABLE events (
    id         BIGSERIAL PRIMARY KEY,
    name       TEXT        NOT NULL,
    venue      TEXT        NOT NULL,
    starts_at  TIMESTAMPTZ NOT NULL,
    created    TIMESTAMPTZ NOT NULL DEFAULT now(),

    CONSTRAINT name_not_blank CHECK (length(trim(name)) > 0)
);
```

The three `DROP` lines are in that order on purpose. `bookings` points at `seats`, which points at `events` — so you take them down from the outside in. Drop `events` first and Postgres refuses, because rows would be left pointing at nothing.

Then the first new thing. `NOT NULL` stops `name` being absent. It does **not** stop it being `""` or `"  "`, which is an empty event name with extra steps. `CHECK` closes that: a condition that must be true for the row to be allowed to exist.

Seats — where most of the rules live

```
CREATE TABLE seats (
    id        BIGSERIAL PRIMARY KEY,
    event_id  BIGINT NOT NULL REFERENCES events(id) ON DELETE CASCADE,
    seat_no   INT    NOT NULL,
    status    TEXT   NOT NULL DEFAULT 'available',

    CONSTRAINT status_is_known       CHECK (status IN ('available', 'booked')),
    CONSTRAINT seat_no_is_positive   CHECK (seat_no > 0),
    CONSTRAINT one_seat_no_per_event UNIQUE (event_id, seat_no)
);

CREATE INDEX seats_event_status_idx ON seats (event_id, status);
```

Four rules in there, and each one is a bug you now cannot write.

1. `REFERENCES events(id)` — a foreign key

This column must hold the `id` of a row that actually exists in `events`. Not a number — an existing event. A seat for event 999 is not a mistake you have to catch; it is a row Postgres will not store.

2. `ON DELETE CASCADE`

The foreign key raises a question: you delete an event, so what happens to its 400 seats? Postgres will not leave them pointing at nothing, so it needs an instruction. `ON DELETE CASCADE` is "they go too." Measured, on an event with three seats:

measured

```
seats before deleting the event   -> 3
seats after  deleting the event   -> 0
```

One `DELETE` statement from you, and the seats are gone as well. The alternative — deleting children by hand in your service — is a loop you can forget to write.

3. `CHECK` on status

A seat is `'available'` or `'booked'`. Without this line, `'avaliable'` is a perfectly good string and your seat listing silently stops finding it. One typo, no error, wrong page.

4. `UNIQUE (event_id, seat_no)` — a composite unique

Read it as a pair, not two rules. It says the **combination** must be unique. So:

measured

```
seat_no 1 in event A, and seat_no 1 in event B  -> allowed
seat_no 1 twice in event A                      -> UniqueViolationError
```

Which is exactly right. Every event has a seat 1. No event has two of them. A plain `UNIQUE (seat_no)` would have meant only one event in the whole system could ever have a seat 1.

The `CREATE INDEX` line underneath is not a rule, it is speed: you are about to run `WHERE event_id = ... AND status = ...` on every seat page, and this stops Postgres reading the whole table to answer it.

Bookings

```
CREATE TABLE bookings (
    id          BIGSERIAL PRIMARY KEY,
    seat_id     BIGINT      NOT NULL REFERENCES seats(id) ON DELETE CASCADE,
    user_email  TEXT        NOT NULL,
    created     TIMESTAMPTZ NOT NULL DEFAULT now(),

    CONSTRAINT one_booking_per_seat UNIQUE (seat_id)
);
```

Small table, and one line in it matters more than anything else you write today: `UNIQUE (seat_id)`.

It says a seat can appear in the bookings table once. Not once per user, not usually — once. Two rows for the same seat is a thing Postgres is physically unable to store, no matter what your Python does.

> **Remember this line tomorrow.** Tomorrow you write the booking endpoint, get it wrong, and this constraint is what stands between a wrong endpoint and forty people holding a ticket for the same chair. You will see the measurement.
>
> It is also not enough on its own, and the reason why is the whole of day 2.

Run it, then try to break it

in your terminal; the indented lines are typed inside psql

```
psql postgres

    CREATE ROLE ticket WITH LOGIN PASSWORD 'ticketpass';
    CREATE DATABASE ticketflow OWNER ticket;
    \\q
```

```
psql -U ticket -d ticketflow -f schema.sql
```

Now spend ten minutes in `psql` attacking your own schema. This is the fastest way to believe it. Here is every one of these run against the real database:

WHAT I TRIED	WHAT POSTGRES SAID
a seat for a non-existent event	`ForeignKeyViolationError`
seat_no 1 twice in one event	`UniqueViolationError`
seat_no 1 in event A and in event B	allowed — which is the point of the composite
status set to `'avaliable'`	`CheckViolationError`
`seat_no = 0`	`CheckViolationError`
an event named `' '`	`CheckViolationError`
delete an event with 3 seats	the 3 seats went too

Every one of those is a bug that can no longer reach production, and not one of them cost you a line of Python. That is the trade the schema is making for you.

PART 2 · The folders, now that there are three of everything

Same three layers as project 3. The difference is that there are now three resources, and this is the point where the structure stops being an exercise and starts saving you.

```
ticketflow/
    main.py
    .env
    schema.sql
    pytest.ini
    app/
        config.py
        db.py
        schemas.py
        repositories/
            events.py     bookings.py     seats.py
        services/
            events.py     bookings.py     seats.py
        routers/
            events.py     bookings.py
    tests/
        test_booking.py
```

Plus the empty `__init__.py` in `app/` and in each of the three folders under it, same as project 3.

Notice the shape: one file per resource per layer. When something is wrong with booking, you open three small files and you know which one holds SQL, which one holds rules, and which one holds status codes. In one `main.py` this

project would be around 400 lines and you would be scrolling for all of them.

`config.py`, `db.py` and `.env` are project 3's, word for word, except the database name and a bigger pool:

app/db.py

```
async def make_pool():
    return await asyncpg.create_pool(DATABASE_URL, min_size=2, max_size=20)
```

.env (and into .gitignore)

```
DATABASE_URL=postgresql://ticket:ticketpass@localhost:5432/ticketflow
```

schemas.py

app/schemas.py

```
from datetime import datetime

from pydantic import BaseModel


class EventIn(BaseModel):
    name: str
    venue: str
    starts_at: datetime
    total_seats: int


class EventUpdate(BaseModel):
    name: str | None = None
    venue: str | None = None
    starts_at: datetime | None = None


class BookingIn(BaseModel):
    seat_id: int
    user_email: str
```

`starts_at: datetime` means Pydantic will take `"2026-11-20T18:30:00Z"` from the JSON and hand your function a real Python `datetime`. Rubbish in that field is rejected as a 422 before your code runs — the same deal `int` gave you in project 1.

`EventUpdate` is the new shape. `str | None = None` reads as "a string, or nothing, and nothing is the default." Every field optional, because a `PATCH` is "change only what I send you." Part 3 shows what the SQL does with the fields the user left out.

PART 3 · The events repository — four new pieces of SQL

One change before the code, and it is small but it matters later today.

In project 3 every repository function started `async def insert(pool, ...)`. Here they start `async def insert(conn, ...)`. The reason is that in Part 4 some of these have to run inside a transaction, and a transaction belongs to **one connection**, not to the pool.

> A pool and a single connection both have `.fetch`, `.fetchrow` and `.execute`. So a repository function that only ever calls those does not care which one it is handed.
>
> Call it with the pool when the query stands alone. Call it with a connection when it has to be part of something bigger. The function never changes.

Insert an event

app/repositories/events.py

```
async def insert(conn, name, venue, starts_at):
    return await conn.fetchrow(
        """INSERT INTO events (name, venue, starts_at)
           VALUES ($1, $2, $3)
           RETURNING *""",
        name, venue, starts_at,
    )
```

Nothing new. The triple quotes are just so the SQL can sit on several lines and stay readable — your queries are getting too long for one line now.

400 seats without a Python loop

An event with 400 seats needs 400 rows in `seats`. The obvious way is a loop in Python firing 400 inserts, which is 400 round trips to the database for one request.

Postgres can make the numbers itself:

```
async def insert_seats(conn, event_id, total_seats):
    await conn.execute(
        """INSERT INTO seats (event_id, seat_no)
           SELECT $1, generate_series(1, $2)""",
        event_id, total_seats,
    )
```

`generate_series(1, 400)` produces the numbers 1 to 400 as rows. The `SELECT` pairs each one with the same `event_id`, and `INSERT ... SELECT` writes whatever that `SELECT` produced. One statement, one trip, 400 rows.

The JOIN — how many seats are left

Your events list should say “Spring Fest, 312 seats left.” But `events` does not have a seats column, and it should not — that number changes every time somebody books. It has to be counted from the `seats` table at the moment you ask.

Reading two tables in one query is a `JOIN`.

```
async def list_with_seats_left(conn):
    return await conn.fetch(
        """SELECT e.*, count(s.id) AS seats_left
           FROM events e
           LEFT JOIN seats s ON s.event_id = e.id AND s.status = 'available'
           GROUP BY e.id
           ORDER BY e.starts_at"""
    )
```

PIECE	WHAT IT MEANS
`FROM events e`	e is a nickname for the table, so you can write `e.id` instead of `events.id`.
`LEFT JOIN seats s ON ...`	Glue each event to its seat rows. `ON` says how they match up: a seat belongs to this event.
`LEFT`	Keep events that match no seats at all. A plain `JOIN` would silently drop a sold-out event from your list.
`GROUP BY e.id`	Squash all the glued-on seat rows for one event back down into one row.
`count(s.id) AS seats_left`	How many got squashed. That is your number.

THE ONE THAT CATCHES PEOPLE

Look where `AND s.status = 'available'` sits. It is in the `ON`, not in a `WHERE`.

In a `WHERE` it would run *after* the join, throwing away every row that is not an available seat — and a sold-out event has none, so the event itself disappears from your list entirely.

In the `ON` it is part of *how they glue together*. A sold-out event glues to nothing, `LEFT` keeps it anyway, and `count` says 0. Which is what you wanted: the event shows up, with zero seats left.

PATCH, and the fields the user did not send

A `PATCH` sends only what is changing — maybe just `venue`. The other two arrive as `None`. You must not write `None` into the name column.

The brute-force fix is building the SQL string differently depending on which fields turned up. Do not. Postgres has one function for exactly this:

```
async def update(conn, event_id, name, venue, starts_at):
    return await conn.fetchrow(
        """UPDATE events
           SET name      = COALESCE($2, name),
               venue     = COALESCE($3, venue),
               starts_at = COALESCE($4, starts_at)
           WHERE id = $1
           RETURNING *""",
        event_id, name, venue, starts_at,
    )
```

`COALESCE(a, b)` means "a, unless it is `NULL`, in which case b." So each column is set to the new value if one was sent, and to **itself** if not. One statement, every combination of fields, no string building. Measured:

measured

```
PATCH /api/events/1  {"venue": "Kalidas Audi"}
  -> name stayed "Spring Fest", venue became "Kalidas Audi"
```

And the two plain ones

```
async def get(conn, event_id):
    return await conn.fetchrow("SELECT * FROM events WHERE id = $1", event_id)


async def delete(conn, event_id):
    return await conn.execute("DELETE FROM events WHERE id = $1", event_id)
```

Straight from project 3. Worth remembering what that `DELETE` now also does: the event's seats, and their bookings, go with it. You wrote that rule once, in the schema.

PART 4 · Two writes that have to happen together

Creating an event is not one write. It is two: the event row, then its seat rows. And they are not independent — an event with no seats is not a half-made event, it is garbage sitting in your database that nobody can book and nobody will ever notice.

So what happens if the second write fails? Bad data in the seat count, Postgres restarting, your laptop lid closing. The first write already happened. Here is that, measured — an event inserted, then a deliberately invalid seat:

measured

```
event row inserted, id = 4
seat insert failed: CheckViolationError
events before: 0,  events after: 0
```

Event 4 existed, and then stopped ever having existed. Nothing cleaned it up and no `except` block undid it. That is a **transaction**.

WHAT A TRANSACTION ACTUALLY IS

A fence around a group of writes, with one rule: **all of them, or none of them.**

Inside the fence the writes are real to you — you can read back the id you just inserted. They are not real to anyone else yet. At the end, one of two things happens: the fence closes cleanly and everything becomes real at once (*commit*), or something raised, and every write inside is erased as if it never ran (*rollback*).

There is no middle. That is the entire idea, and it is why the word "atomic" gets used — the group cannot be cut in half.

In the service

The transaction goes in the service, not the repository — it is a decision about how two pieces of work relate, and that is what the service layer is for.

app/services/events.py

```
from app.repositories import events as repo


class NotFound(Exception):
    pass


class BadSeatCount(Exception):
    pass


async def create(pool, name, venue, starts_at, total_seats):
    if total_seats < 1 or total_seats > 1000:
        raise BadSeatCount()

    async with pool.acquire() as conn:
        async with conn.transaction():
            event = await repo.insert(conn, name, venue, starts_at)
            await repo.insert_seats(conn, event["id"], total_seats)

    return event
```

LINE	WHAT IT DOES
`pool.acquire()`	Borrow one connection out of the pool and keep it. A transaction lives on a single connection, so you need the same one for both writes.
`conn.transaction()`	Open the fence. Everything until the block ends is one group.
the two `repo` calls	Both get `conn`, not `pool` — which is why the repository took `conn` all along.
leaving the inner block	No exception: commit, both writes land. An exception: rollback, both vanish, and the exception carries on up to you.
leaving the outer block	The connection goes back to the pool for the next request.

You never write `commit` or `rollback`. Reaching the end of the block is the commit; raising out of it is the rollback. That is what `async with` is doing for you.

The rest of the service

```
async def listing(pool):
    return await repo.list_with_seats_left(pool)


async def get_one(pool, event_id):
    row = await repo.get(pool, event_id)
    if row is None:
        raise NotFound()
    return row


async def edit(pool, event_id, name, venue, starts_at):
    row = await repo.update(pool, event_id, name, venue, starts_at)
    if row is None:
        raise NotFound()
    return row


async def remove(pool, event_id):
    result = await repo.delete(pool, event_id)
    if result == "DELETE 0":
        raise NotFound()
```

These four are project 3 exactly — and notice they hand over `pool`, not a connection, because each is a single statement that stands on its own. Only `create` needed the fence.

One nice thing falls out of `COALESCE`: `edit` gets `None` back when the id does not exist, because `UPDATE ... RETURNING *` on zero rows returns nothing. Same `None` check as a `SELECT`.

Seats get a tiny service of their own

app/services/seats.py

```
from app.repositories import events as events_repo
from app.repositories import seats as repo
from app.services.events import NotFound


async def listing(pool, event_id, status):
    event = await events_repo.get(pool, event_id)
    if event is None:
        raise NotFound()
    return await repo.list_for_event(pool, event_id, status)
```

It checks the event exists first, so asking for the seats of event 999 is a clean 404 instead of an empty list that looks like a sold-out show. A service is allowed to call more than one repository — that is the layer where things get combined.

app/repositories/seats.py

```
async def list_for_event(conn, event_id, status):
    return await conn.fetch(
        """SELECT * FROM seats
           WHERE event_id = $1
             AND ($2::text IS NULL OR status = $2)
           ORDER BY seat_no""",
        event_id, status,
    )
```

That second condition is the optional filter. If no `status` was sent it arrives as `NULL`, the left half is true, and the whole bracket is true for every row — the filter switches itself off. If a status was sent, the right half does the work.

The `::text` is Postgres asking what type $2 is meant to be, since `NULL` on its own does not say. One small annotation, and you avoid writing two different queries.

PART 5 · The router, and running it

One new thing here, and it is a convenience: `APIRouter` takes a `prefix`.

```
router = APIRouter(prefix="/api/events")
```

Every path on this router now starts with that, so you write `@router.get("")` for `/api/events` and `@router.get("/{event_id}")` for `/api/events/1`. The empty string is not a typo — the prefix is the whole path.

app/routers/events.py

```
from fastapi import APIRouter, HTTPException, Request

from app.schemas import EventIn, EventUpdate
from app.services import events as service
from app.services import seats as seats_service

router = APIRouter(prefix="/api/events")


@router.post("", status_code=201)
async def create_event(body: EventIn, request: Request):
    try:
        row = await service.create(
            request.app.state.pool,
            body.name, body.venue, body.starts_at, body.total_seats,
        )
    except service.BadSeatCount:
        raise HTTPException(status_code=400, detail="total_seats must be between 1 and 1000")
    return dict(row)


@router.get("")
async def list_events(request: Request):
    rows = await service.listing(request.app.state.pool)
    return [dict(r) for r in rows]


@router.get("/{event_id}")
async def get_event(event_id: int, request: Request):
    try:
        row = await service.get_one(request.app.state.pool, event_id)
    except service.NotFound:
        raise HTTPException(status_code=404, detail="Event not found")
    return dict(row)
```

Catch the service exception, raise the HTTP one. Same move as project 3, five times over. Nothing in this file knows any SQL.

```
@router.patch("/{event_id}")
async def update_event(event_id: int, body: EventUpdate, request: Request):
    try:
        row = await service.edit(
            request.app.state.pool, event_id, body.name, body.venue, body.starts_at
        )
    except service.NotFound:
        raise HTTPException(status_code=404, detail="Event not found")
    return dict(row)


@router.delete("/{event_id}", status_code=204)
async def delete_event(event_id: int, request: Request):
    try:
        await service.remove(request.app.state.pool, event_id)
    except service.NotFound:
        raise HTTPException(status_code=404, detail="Event not found")


@router.get("/{event_id}/seats")
async def list_seats(event_id: int, request: Request, status: str | None = None):
    try:
        rows = await seats_service.listing(request.app.state.pool, event_id, status)
    except seats_service.NotFound:
        raise HTTPException(status_code=404, detail="Event not found")
    return [dict(r) for r in rows]
```

The last one is a path with a parameter in the *middle* — `/api/events/1/seats`. Nothing special about it; FastAPI reads `{event_id}` wherever it sits. `status: str | None = None` is an optional query parameter, which is why no status at all is a valid request.

main.py

```
from contextlib import asynccontextmanager

from fastapi import FastAPI

from app.db import make_pool
from app.routers import events


@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.pool = await make_pool()
    yield
    await app.state.pool.close()


app = FastAPI(lifespan=lifespan)
app.include_router(events.router)
```

Identical to project 3. Tomorrow one more `include_router` line joins it.

```
uvicorn main:app --reload
```

What you should see

All measured, against the real thing:

measured

```
POST   /api/events              201   {"id":1,"name":"Spring Fest","venue":"Netaji Audi",...}
POST   total_seats = 0          400   {"detail":"total_seats must be between 1 and 1000"}
GET    /api/events              200   [{"name":"Spring Fest",...,"seats_left":5}]
GET    /api/events/1/seats      200   seat_no 1,2,3,4,5 all "available"
GET    .../seats?status=booked  200   []
PATCH  /api/events/1  venue     200   name unchanged, venue changed
GET    /api/events/9999         404   {"detail":"Event not found"}
DELETE /api/events/1            204
DELETE /api/events/1  again     404   {"detail":"Event not found"}
```

If you get all nine of those, Saturday is done.

PART 6 · Break it on purpose

DO THIS	AND NOTICE
In `psql`, `INSERT INTO seats (event_id, seat_no) VALUES (999999, 1)`	Foreign key violation. Your API never had a chance to be wrong about this.
Change `LEFT JOIN` to `JOIN`, then book out every seat of an event	The event disappears from `GET /api/events` completely. Change it back.
Move `AND s.status = 'available'` from the `ON` into a `WHERE`	Same disappearance, subtler cause. This is the Part 3 marker, felt rather than read.
Delete the `COALESCE`s and PATCH only `venue`	`name` and `starts_at` get wiped to null — or the `NOT NULL` stops you. Either way you see why it was there.
Inside `create`, add `raise RuntimeError()` between the two repo calls	500 as expected — then check `psql`. No half-made event. The fence held.
Now take the `async with conn.transaction():` line out and do it again	An eventless-seatless event, sitting in your table. This is the one worth doing.
Create an event with 400 seats	One request. Then `SELECT count(*) FROM seats` — 400 rows, no Python loop.

Where Saturday leaves you

A working events API, and a database that refuses to hold bad data. Three tables with real rules, a JOIN that counts across them, and two writes that cannot be cut in half.

And one thing left open.

You have a seat, and it has a status, and the status can be `'booked'`. But nothing in anything you wrote today actually books it.

Tomorrow that endpoint takes about fifteen minutes to write, and it will look obviously correct. Read the seat, check it is free, mark it booked, store the booking. You already know every line of it.

Then two people press the button in the same millisecond, and you find out what that `UNIQUE (seat_id)` line was really for — and why it is not enough.

Three lines for your notes

```
constraints = rules Postgres enforces, so Python does not have to
LEFT JOIN + condition in ON, never in WHERE
transaction = all of the writes, or none of them
```